Course architecture

This document defines the **intellectual purpose and planned progression** of the course. It guides scope choices and explains how mathematical understanding supports competence with actual problems. The student syllabus summarizes it; the curriculum map allocates proposed topics and practical exercises; agreed lecture scopes own detailed commitments. The actual curriculum map will record what is taught once classes begin. Planned and actual coverage should be compared and reconciled without turning prepared materials into claims about student learning.

What the course is for

The goal is **working competence in probabilistic machine learning**. Since the field of machine learning is large and constantly developing, it is not the goal of this course to be exhaustive. Rather, the student should have practical experience with a number of substantive datasets, the types of scientific and practical questions that can be asked about them, and the computational tools that are appropriate. This practical knowledge is built upon a **coherent probabilistic understanding** of the field. The probabilistic approach is the main intellectual instrument for developing the practical competence. By understanding machine learning problems as a process of formulating a probabilistic model, specifying the desired query on that probabilistic model, and a performance criterion for evaluating the answer, students will have a tool for extending their competence to new problems. The computational methods are seen as tools appropriate for exploiting specific structure and scale of the problem at hand and its formulation. The mathematical details matter: without them, students cannot see what an algorithm computes, why its assumptions help, when it fails, or how it differs from an alternative. This is a graduate course in which students already know probability and Bayesian statistics; it develops that background into the ability to tackle new machine-learning problems.

The course therefore has **two intertwined threads**. One develops the mathematical ability to formulate and compare methods. The other builds a concrete repertoire: particular learning problems, datasets, implementations, performance checks and diagnoses. Every substantive lecture should advance both threads at an appropriate depth. The algorithm or mathematical principle is an **exemplar** of a transferable idea; the example is substantial enough that students learn how to use and question the method, rather than merely recognize its algebra. Practical competence here means credible, guided first use and informed method selection—not unassisted production mastery of every technique after one notebook.

From learning problems to mathematical formulations

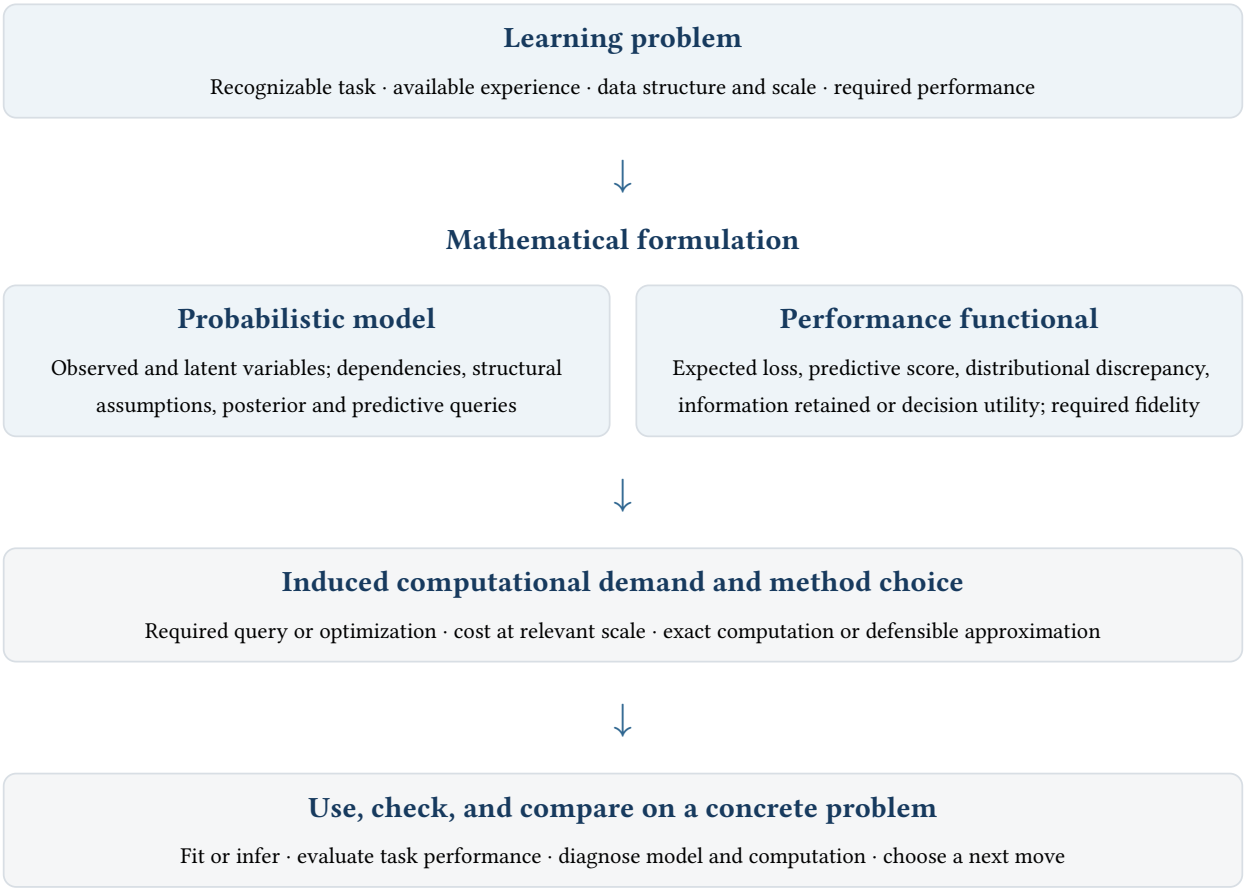
The first lecture formulates the language and the approach in which the rest of the course will understand machine learning. A **learning task** specifies what a system should accomplish; its **experience** is the data, feedback, or interaction available. The **structure and scale** of that experience matter. Its success must be judged by a **performance requirement**: what predictions, distributions, representations or decisions must be good, and how good is good enough? The same dataset can support prediction, generation, discovery or action, each with different legitimate criteria.

The mathematical formulation contains **two distinct, related objects**. A probabilistic model represents relevant observed and latent variables, their dependencies, and uncertainty. A performance functional specifies what counts as a good answer. For example, a predictive action a can be evaluated by

$$\mathcal{R}(a) = \int L(a, y) p(y) \, \mathrm{d}y.$$

The distribution under which actual performance is assessed need not equal the model we fit. Model mismatch, the choice of training surrogate, and numerical or approximation error are different possible sources of failure. A formulation gives us **probabilistic queries**: conditional distributions, marginal probabilities, expectations, predictions or decisions, often requiring integrals or optimizations. This means, directly, that the same data and probabilistic formulation can be used for different queries requiring, potentially, different computational approaches. This is established in the first lecture, and the rest of the course develops both richer probabilistic models and ways to perform the necessary computations at useful scale. A student's practical question is not merely "Which formula applies?" but "Which answer do I need, how can I compute it, and how can I tell whether it is good enough?"

The task–model–performance–computation relationship



A reasoning framework, not a rigid recipe: failures in application can lead us to revise the task, model, criterion or algorithm.

The course's main mathematical **moves** broaden **(i) what probabilistic models can express, (ii) what performance functionals can distinguish, and (iii) what computational methods can deliver**. The computational obstacle is **induced** by the data structure and size, the model and query, the performance requirement, and available resources. An algorithm becomes appropriate because it meets those particular demands, not because the syllabus has reached its chapter.

The ideal quantity and the practical calculation

Computational difficulties can concern the probabilistic query, fitting the model, or evaluating performance; these are not necessarily the same calculation. A factorization may let us reorganize a sum and obtain an **exact** result more cheaply. A point estimate may replace integration over parameter uncertainty; Laplace or variational inference may instead supply an approximate posterior distribution; and Monte Carlo may estimate a particular expectation using a finite sample. A desired performance criterion may itself be estimated on data or replaced by a tractable training surrogate. Some of these moves simplify computation without approximating the target,

while others introduce specific errors. We should not call all of them "posterior approximations." For any chosen approach, the recurring questions are: **When is it appropriate for this structure and scale? What computation does it save relative to a plausible alternative? Is its accuracy sufficient for the actual task?**

How mathematical depth becomes practical familiarity

Each lecture needs a recognizable **problem encounter**: what is observed, what must be predicted, inferred, generated or decided, and how success will be checked. Its mathematical argument then explains a method's assumptions, what is computed and the regime in which the method pays off. Finally, students should see and use the approach on a concrete dataset, inspect its output and failures, and compare it with at least one meaningful baseline or alternative when feasible. This is a **design test**, not a mandatory slide sequence; some lectures develop representation or performance rather than a new algorithm.

A transparent toy model and a credible practical example have **different instructional jobs**. A small controlled case can expose a theorem, exact reference, bias or numerical failure that is otherwise hidden. A substantive case demonstrates why one would use the method and what it means to evaluate an answer. A lecture or notebook may use both, with much of the code or data preparation supplied, rather than forcing every student to implement two complete applications in an hour. Whenever possible, revisit an earlier problem with a new method: the difference between tools becomes visible when the question, dataset and performance criterion stay fixed.

The approximately two-hour lecture develops the mathematical reasoning, conceptual comparisons and interpretation of representative results, often through a worked example. The approximately one-hour notebook provides a guided encounter with the method: **construct or run** it, **read and validate** its output, and **judge** whether it answers the learning problem. The expected effort is in recognizing and using ideas, not in independently building a substantial program. Questions should be local, short, fully specified, and easy to answer with the information and code already provided; unfamiliar concepts and APIs need gentle introduction, while familiar ones can be exercised without unnecessary re-teaching.

The course's notebook pattern is a fine-grained, AI-assisted question-and-answer sequence, not an open-ended software assignment. Authors first validate a worked solution, then write a solved notebook with questions followed immediately by complete answers; the student version is generated from that solved source. Supply data preparation, plotting helpers and arbitrary setup choices when they are not the point of the lesson. Small steps can include interpreting a mathematical object, applying a lecture equation, completing a short operation, predicting or checking an output, and evaluating a meaningful comparison. Students may consult AI for each self-contained question. The questions should be easy to approach without making the reasoning entirely optional. Use the prepared L01 solved notebook and the Sleep Deprivation notebook sequence as style precedents, adapted to this course's methods. The notebook-building and teaching-notebook skills define the detailed authoring and validation process.

Murphy's two books provide the main mathematical and conceptual source. Book 2 supplies the broader logic and methods; Book 1 often supplies more gradual derivations or useful introductory examples. The course is a deliberate selection, not comprehensive chapter coverage. Formal proofs and calculations are retained where they explain a method's ability, limits, or relation to other methods. Their pedagogical value is tested by the understanding and problem-solving judgment they make possible—not by the number of pages or results reproduced.

How the semester develops

What constitutes a class of learning problems?

A learning-problem family is not simply "medicine," "images," or a convenient dataset on which to demonstrate an algorithm. It is characterized by **the available experience and the structure and size of the data** (what is observed, missing, sequential, dependent, or high-dimensional), **the task and probabilistic question**, and **the performance requirement** (what must be estimated, predicted, preserved or decided, and how accurately). These induce the necessary calculations and their costs relative to a computational budget. **Computational difficulty is a consequence of the formulation, not a fourth independent ingredient.** Different datasets may share the same mathematical problem, while the same dataset may support several distinct questions.
But the purpose of this classification is **to help students solve recognizable machine-learning problems**. Every lecture therefore needs a practical destination as well as an intellectual contribution: students should understand **what the problem looks like in data**, which method they would reach for, what its output means, how it is evaluated, and where it is likely to fail. The mathematical progression supplies a shared explanation for that repertoire. A lecture fails if students see only formulas without a usable technique, or only a technique without an intelligible reason for choosing it.
The semester follows a recurring argument: **recognize the learning problem → formulate its model and performance → identify the required computation and its scale → develop the mathematical move → use and evaluate the method**. This is a conceptual relationship, not a required presentation order. The notes and notebooks may begin with a practical puzzle or a mathematical result, whichever reveals the connection most clearly. We should cultivate both familiarity with individual methods and the ability to transfer that familiarity to a new dataset.

Models, probabilistic queries and performance (L01–L04)

L01 — Probabilistic machine learning. When measurements are incomplete or noisy, the same dataset can support prediction of a missing measurement, inference of a hidden state, generation or a decision. The practical starting point is a multivariate measurement problem: students specify what they observed, what they want, and what counts as a useful answer; then use multivariate Gaussian conditioning to impute missing observations and quantify uncertainty. The **mathematical move** is to express different tasks as conditioning, marginalization and summarizing a joint distribution. The Gaussian case provides a complete, workable baseline; it also establishes the challenge of retaining this problem-solving ability when the relevant distributions stop being Gaussian. The prepared L01 notebook is designed to give initial practical experience with conditioning, rather than just a derivation of conditional moments; this has not yet been tested in class.
L02 — The exponential family. Binary measurements, real measurements, categories, counts and observed character transitions each call for an appropriate distribution. Students should recognize these data types, estimate a law from observed samples, score new cases and simulate additional examples. Exponential-family models make the connection between these tasks transparent: sufficient statistics summarize relevant data; derivatives of the log-partition function give moments; moment matching supports MLE; maximum entropy motivates distributions under stated constraints. The **practical exemplar** is a character bigram model: count transitions, fit and inspect the transition matrix, score strings under the fitted model and generate names; held-out evaluation is optional. The goal is not to reduce text modeling to bigrams, but to learn a complete small modeling cycle and recognize when the Markov/finite-statistic assumptions would be inadequate.
L03 — Graphical models An image, network or spatial measurement field is not well modeled as a collection of unrelated observations. Students should learn to recognize a plausible conditional-independence structure,

specify its factors, and use a graphical model to predict or reconstruct missing components. DAGs, d-separation, Markov blankets and undirected potentials express different constraints on a joint probability distribution; the reduced representation may, but need not, make inference tractable. In the image-restoration exemplar, students construct an Ising prior and an observation likelihood, derive a single-site conditional, and perform a local reconstruction update. They must interpret the image and recognize that conditional optimization is not global MAP or a posterior marginal. The **transferable insight** is when a structural assumption enables modeling and which further computational question remains open for L08.

L04 — Information theory. A classifier can be correct but badly calibrated; a compressed representation can preserve some distinctions while losing others. These are learning problems whose success cannot be described by a single accuracy or squared-error number. The **performance move** introduces entropy, cross-entropy, KL divergence and mutual information. Expected negative log score relates probabilistic predictions to KL discrepancy; mutual information describes a selected dependency, not universal usefulness for every decision. Students should compare **actual probabilistic predictions**—for instance, prepared competing probability forecasts for Iris species—and identify what different performance measures reward. The noisy parity-code example offers an exact small experiment in the tension between information, error probability and resource rate, but it is **not the practical destination of the lecture by itself**. It should be explicitly related back to evaluation of machine-learning predictions and learned representations. The notebook should make a real distinction between a metric calculated correctly and a model or decision that is useful.

Fitting probabilistic models (L05–L06)

These lectures extend fitting beyond the elementary models of L01–L02. GLMs specify appropriate conditional response distributions and enable direct gradient-based point fitting; EM uses conditional expectations to fit models with missing or latent observations. Neither should be confused with obtaining a full posterior over model parameters.

L05 — Generalized linear models. Predicting a species or a binary clinical outcome requires different observation distributions from predicting a continuous measurement; counts introduce another mean–variance relationship. GLMs extend the repertoire of conditional likelihoods, with links relating their parameters to predictors. For canonical links, their structured gradients and convex likelihoods (under appropriate conditions) make point estimation especially manageable, although full parameter-posterior inference may not be. The **computational move** is to use likelihood gradients, reverse-mode automatic differentiation and minibatch optimization for an MLE/MAP fit. Students fit and evaluate a softmax classifier on Iris, inspect its predicted probabilities and held-out score, and compare hand-derived gradients with those from software. The aim is familiarity with an end-to-end probabilistic classifier, not just a backpropagation identity. The model's predictive output, training objective and empirical performance must remain distinct.

L06 — Expectation Maximization. Suppose several physiological measurements are missing for some participants, or an observation belongs to an unknown mixture component. With unknown population parameters, imputing a value is no longer the whole task: we need to **fit the model using the observed data**. Marginal likelihood integrates over latent variables; filling each one in with a point guess changes the objective. Expectation–maximization exploits a tractable conditional posterior and manageable expected complete-data statistics to alternate inference with fitting. The ELBO/KL identity explains its likelihood-improvement guarantee under exact, improving steps. Gaussian incomplete-data fitting needs expected second moments **including the conditional covariance**, connecting L01′s Gaussian calculations to L02′s sufficient statistics. Practically, students fit parameters under controlled missingness, compare them with complete-case or mean-imputed

baselines, and distinguish observed-data likelihood from reconstruction quality. The technique is worthwhile only if its E- and M-steps are tractable; EM supplies a point fit, not full parameter-posterior uncertainty.

Inference: exact computations and defensible approximations (L07–L10)

The methods in this block solve related but different computational problems. Laplace and VI construct posterior proxies by different criteria; message passing can exploit structure for exact probabilistic sums; MCMC estimates target expectations using dependent samples. They are not a ladder of increasingly advanced approximations. The choice depends on model structure, the query, relevant dimensions and scale, computational resources and acceptable error. Reusing a prediction problem across methods can make the tradeoffs tangible without abandoning small exactly checkable examples.

L07 — Laplace approximation. In Bayesian logistic prediction, a fitted coefficient vector alone does not express parameter uncertainty or posterior-averaged predictive probabilities. If the posterior is smooth, regular and strongly concentrated near a well-identified mode, Laplace constructs a Gaussian proxy from the mode and inverse Hessian. We can then compute or sample from its approximate marginals and predictions without the original posterior integrals. The computational payoff is strong when the parameter dimension and Hessian cost are moderate; neglected modes, boundaries, asymmetry and important tails may invalidate a downstream decision even if the fit looks good near its peak. Students should **use** the approximation to predict with uncertainty, not just compute a curvature. The one-parameter logistic example remains a transparent comparison with checked quadrature, but should be connected to an actual small, overlapping binary-classification problem and a point-prediction baseline.

L08 — Message passing algorithms. Given noisy observations over time, a practical question is whether the underlying system changed state, when, and with what uncertainty. An HMM expresses conditional independence through local transitions and emissions. Filtering, smoothing, sequence evidence and the most probable whole path are distinct answers. Forward–backward dynamic programming rearranges sums over K^T possible histories into reusable local messages, taking $O(TK^2)$ work for dense finite-state transitions. This is exact for the stated model, unlike a posterior-family approximation. Students should infer and interpret state probabilities on a sequence and distinguish an uncertain boundary from a forced segmentation; a tiny enumerated case checks the code. A behavioral-state or sensor-sequence interpretation gives the algorithm a recognizable use, while the casino HMM remains an accessible mathematical testbed. The linear-Gaussian Kalman comparison shows another structural regime permitting exact filtering.

L09 — Variational inference. When repeated posterior queries are costly, the Hessian is unwieldy, or a local Gaussian is not accurate enough for the quantities of interest, we can choose a tractable family of posterior approximations and **optimize a distributional objective**. The ELBO/reverse-KL principle fits the entire candidate distribution rather than matching only local mode and curvature. VI does not by definition require Gaussianity or unimodality. Mean-field or Gaussian approximations gain cheap computation but can lose correlations, tails or modes; flexible families may preserve more at greater cost. Students should be able to fit a simple variational distribution, compare uncertainty and predictions to a checked reference or Laplace baseline, and tell **family restriction** from **optimizer error**. The correlated-Gaussian test problem makes that diagnosis exact; a linked Bayesian prediction application should show how the method is actually used. Sampling to optimize the ELBO or from the fitted proxy is not what distinguishes VI from Laplace.

L10 — Monte Carlo methods. Some posterior questions depend on correlations, rare regions or multiple modes, and an easily fitted Gaussian or variational proxy may miss what matters. If a target density can be evaluated up to normalization, MCMC constructs transitions whose invariant law is that target; sample averages approximate **specific** probabilities, risks and predictions. This does not impose a fixed posterior-family

approximation, but finite-run mixing and autocorrelation can dominate the accuracy and computational cost. Metropolis–Hastings provides the core construction; Gibbs shows the special role of tractable conditionals. Students should **run, diagnose and interpret** a chain for a posterior query, compare estimates across chains and with a baseline, and explain why invariant transitions need not give good finite answers. The bimodal reference distribution exposes poor exploration; applying the same question to a posterior previously approached by Laplace or VI would provide practical comparative judgment.

Expressive prediction and generation (L11–L12)

These lectures revisit, in compact form, Murphy's distinction between **Prediction** and **Generation**. A neural network can parameterize a flexible conditional distribution for prediction (L11), while a VAE couples a latent generative model with learned, amortized inference (L12). Compared with simpler models, the added expressiveness can help with complex data but makes fitting, evaluation and uncertainty harder. The mathematical objective is to understand these tradeoffs; the practical objective is to use and assess a model whose flexibility earns its computational and statistical costs.

L11 — Deep neural networks. For many datasets, an affine GLM or constant-variance response model cannot express the shape or conditional variability of observations. A neural network can parameterize a likelihood through nonlinear functions, such as a Gaussian with input-dependent mean and scale. Reverse-mode autodiff and minibatch training still work for the composed computation, although the optimization is generally nonconvex. Students should train a small conditional model, compare it against a meaningful fixed-scale or simpler baseline on held-out cases, and interpret log score, mean error and predictive intervals. A credible analysis distinguishes observation noise conditional on fitted weights from uncertainty about the weights: a fitted scale is not a Bayesian posterior. The synthetic heteroskedastic test case exposes known truth; its role is to teach whether added expressiveness helps the practical prediction task, not simply that a network can fit an arbitrary curve.

L12 — Variational auto encoders. An unlabeled collection of high-dimensional observations creates different tasks: generating new plausible cases, reconstructing observed cases, and learning compact representations. A flexible latent generative model $p(z)p_{\theta(x|z)}$ can model such data, but separate posterior inference for every new observation is costly at scale. A shared encoder $q_{\varphi(z|x)}$ **amortizes** VI: training is expensive, but applying the inference map to many related cases can be cheap. The reconstruction/rate tradeoff reflects the required task performance; average encoder-to-prior KL is **not** identical to mutual information. Students should train or complete a compact VAE, generate from the prior, reconstruct actual examples, inspect latent codes and compare at least two fitted tradeoffs on meaningful measurements. They should also recognize posterior collapse, inference-family or amortization error, and nonidentifiability: an interpretable plot is not evidence of recovered causal factors. This is a concrete generative-modeling skill and a synthesis of L04, L06, L09 and L11, not merely another application of the ELBO.

Using the tools to address a new problem (L13)

L13 — Demonstrate practical selection and justification on a new problem. Student presentations should begin with a recognizable dataset and problem, identify the task and performance requirement, select a plausible probabilistic model and method, and show an actual result with a meaningful check or comparison. Mathematical explanations should clarify what their algorithms deliver and what remains uncertain. The course's success is visible when students can use a concrete method **and** give a principled reason for choosing or rejecting it—not when they can only recite the unified architecture or reproduce a worked notebook.

Curricular selection and teaching records

The course is a deliberate selection from Murphy, not comprehensive coverage. It retains mathematical depth where a result explains a method's construction, efficiency, validity or limitations, while making room for meaningful practice. Exact sequence inference, Laplace, VI and MCMC are alternatives in different regimes rather than stages of a single approximation hierarchy. More advanced optimization, particle methods, Gaussian processes, Bayesian neural networks and further generative families are worthwhile subjects but have no dedicated teaching commitment in the current thirteen-meeting plan.

The curriculum map gives the proposed allocation of topics, Book 1 and Book 2 reading sections, examples, notebook references and specific omissions. Approved lecture scopes may differ from newly proposed examples; that disagreement is to be considered before modifying a scope. The parallel actual curriculum map starts empty and will be updated after each meeting to capture substantive lecture and student-notebook coverage. Prepared decks and solutions are evidence for planning, not evidence that students encountered or completed material.